

Multi-hop Reasoning in Neural, Symbolic, and Hybrid Models

Sankalp B C^{1†}, Sk Abdur Razaaq^{1†}, Vathsala H^{2*†},
Dinesh Naik^{1†}, Ch. Janaki^{2†}, Sathyanarayana K B^{1†},
S D Sudarsan^{2†}

¹Department of Information Technology, National Institute of Technology, Surathkal, Mangalore, 575025, Karnataka, India.

²Centre for Development of Advanced Computing (C-DAC), Bangalore, 560036, Karnataka, India.

*Corresponding author(s). E-mail(s): vathsalah@cdac.in;

Contributing authors: sankalpbeerappa.253it002@nitk.edu.in;

skabdurrazzaq.253it001@nitk.edu.in; din_nk@nitk.edu.in;

janaki@cdac.in; sathyanarayanakb.2375036@nitk.edu.in; sds@cdac.in;

[†]These authors contributed equally to this work.

Abstract

Imagine a situation where an automated eligibility system has to determine whether a claimant is the legal heir of a deceased person, by proving that the claimant’s father’s mother is the deceased person’s sister. To reach this conclusion, the system must sequentially link three independently stated facts. This capacity is referred to as “multi-hop reasoning,” and it continues to be a major obstacle for artificial intelligence (AI). AI’s reasoning accuracy declines drastically as the reasoning chain expands in multi hop statements. Modern AI language models are excellent at pattern recognition, however, it lacks systematic thinking. Although classical rule-based systems provide logical accuracy, they cannot handle the ambiguity of natural language. Thus, current systems are unsuited for high-stakes deployment, as no existing approach simultaneously guarantees logical consistency, compositional generalization, and human-readable explanations over arbitrary reasoning depths. This work proposes a modular neural-symbolic architecture called **Hybrid Reasoner**. It uses a refined Bidirectional Encoder Representations from Transformers (BERT) model that extracts structured facts from natural language, and also employs deterministic First-Order Logic (FOL) engine for forward chaining over Horn clauses, this ensures connecting the facts into guaranteed, step-by-step conclusions making inference formally tractable. A confidence-based orchestrator is employed to select between

the results of symbolic path and the neural module. The proposed model is assessed using the Compositional Language Understanding and Text-based Relational Reasoning (CLUTRR) benchmark dataset. The dataset covers 18 different types of kinship relations with two to ten hops inference chains. The model is evaluated by accuracy, Macro/Micro F1, and logical consistency; the proposed model, **Hybrid Reasoner** achieves exceptional logical consistency and accuracy on symbolically tractable queries, it exhibits nearly constant performance across all reasoning depths, establishing a tangible step toward reliable and understandable AI.

Keywords: Neural-Symbolic AI, Multi-hop Reasoning, Relational Inference, First-Order Logic, Explainable AI, Hybrid Reasoning

1 Introduction

In order to establish legal kinship for inheritance purposes and determine that a claimant’s father’s mother is the deceased’s sister, an automated reasoning system must chain three independently stated facts sequentially without explicitly stating the conclusion in a single sentence. A wide range of significant real-world tasks, such as determining kinship-based legal eligibility, resolving hereditary medical risk across generations, validating automated genealogical records, and assisting clinical decision systems that track familial disease patterns, rely on this ability, known as **multi-hop reasoning**. Building systems that can consistently connect facts across numerous inference steps—the way a human expert does with ease—remains a challenge despite notable advancements in artificial intelligence (AI). This is still one of the most enduring unresolved issues in the domain of computing.

There are two different paradigms for solving this issue. Symbolic AI systems may generalize arbitrarily deep inference chains without experiencing performance deterioration since they are based on formal logic and rule-based inference, thereby attaining perfect logical consistency [1]. Transparent, auditable reasoning traces are produced by learning-based extensions like Inductive Logic Programming (ILP) [1] and classical logic programming systems. However, they need hand-crafted rule sets that do not scale elegantly for open-domain input and are infamously brittle when faced with the noise, paraphrasing variety, and pronoun ambiguity inherent in spoken language.

In contrast, neural techniques are remarkably flexible when it comes to handling linguistic variety. Transformer-based models that perform well on language understanding tasks include BERT [2], Graph Neural Networks (GNNs) [3], and Relation Networks [4]. Nevertheless, their accuracy drastically declines as reasoning depth increases—from 95.2% at 2 hops to 62.1% at more than 7 hops on the CLUTRR benchmark [5]—and they mostly depend on statistical pattern matching rather than explicit logical deduction [6, 7]. Their distributed internal representations are not suitable for meaningful post-training interpretation [8, 9], and they produce logically inconsistent outputs under changes in input distribution, achieving only 73.2% logical consistency.

There have been studies that have combined both paradigms, thereby complementing each other in overcoming their respective drawbacks. This approach has been called the neuro-symbolic approach. Previous attempts at neural-symbolic integration include, Knowledge-Based Artificial Neural Networks (KBANN) [10] initialize neural architectures from symbolic structures, however, the interpretability of these structures degrades as fine-tuning proceeds. While DeepProbLog [12] and Neural Theorem Provers (NTPs) [11] render inference differentiable, they forfeit the clear, deterministic nature of traditional logical reasoning. No current method, without imposing differentiability on the logical component, is able to simultaneously ensure logical consistency, compositional generalization over arbitrary reasoning depths, and human-readable proof traces. This three-way gap restricts deployment in high-stakes situations where auditability and correctness are strict requirements.

In order to close this gap, the current work suggests a modular neural-symbolic design called **Hybrid Reasoner**, which treats the two paradigms as complementary experts rather than rivals. The perception module is a refined BERT encoder that extracts structured relational facts from natural language narratives. Then, a deterministic First-Order Logic (FOL) engine employs forward chaining over Horn clauses, this chains the facts into assured, sequential conclusions, generating human-readable proof traces, for each question that is answered. Both modules are coordinated by a confidence-based orchestrator, which only activates the neural module when language comprehension is required and routes each query to the symbolic engine.

This rigorous architectural division is the fundamental innovation of Hybrid Reasoner: neural perception handles the fuzzy, high-variance task of natural language understanding, while symbolic reasoning handles the crisp, depth-invariant task of logical inference—each functioning only in the regime where it is most dependable. A web-based interface (Fig. 2) has been created to show the practical accessibility of this architecture. Hybrid Reasoner takes a relational query and a natural language context narrative, invokes the entire hybrid inference pipeline, and returns a relationship prediction with its supporting proof trace in real time.

The CLUTRR benchmark [5], a compositional generalization testbed containing 18 kinship connection types over inference chains of 2 to 10 hops, is used to assess all three paradigms: neural-only, symbolic-only, and hybrid. Accuracy, Macro F1, Micro F1, and logical consistency are used as assessment criteria to gauge performance.

The rest of this paper is organized as follows. Section 2 surveys related work across neural, symbolic, and hybrid reasoning paradigms. Section 3 presents the Hybrid Reasoner architecture in detail. Section 4 reports experimental evaluation on the CLUTRR benchmark. Section 5 discusses strengths, limitations, and comparisons. Section 6 concludes with directions for future work.

2 Related Work

The objective of developing systems that can think like humans has driven decades of research in the fields of languages, logic, and machine learning. This section traces the intellectual journey from early neural architectures to symbolic systems and, finally,

hybrid paradigms, highlighting how each line of work advanced the field and uncovered new limitations that motivate the present study.

2.1 Neural Approaches to Relational Reasoning

Early attempts to use neural architectures for relational reasoning investigated graph-structured message transmission using GNNs [3] and explicit pairwise comparisons using Relation Networks [4]. Both had potential in completing knowledge graphs and responding to relational questions, however, reasoning was still based on learned embeddings rather than explicit logical rules, which led to poor performance when test-time structural patterns differed from training-time ones.

Marcus [7] argued that statistical pattern matching cannot take the place of rule-based reasoning in tasks requiring logical consistency across multiple inference steps. Lake and Baroni [6] showed that sequence-to-sequence networks fail to generalize compositionally even when constituent operations are individually mastered. These theoretical concerns are experimentally supported by the CLUTRR benchmark [5], where neural accuracy on 7-hop chains drops drastically compared to 2-hop chains [5].

While Ribeiro et al. [9] formalized this fragility by demonstrating that neural classifiers frequently violate logical constraints under input paraphrasing. Lipton [8] noted that distributed representations resist meaningful post-hoc interpretation, limiting trust and debuggability.

All these findings highlight the necessity for a method that preserves logical coherence and produces inference traces that are understandable by humans without sacrificing the linguistic robustness that neural encoders provide.

2.2 Symbolic and Logic-Based Approaches

Symbolic AI addresses the explainability and consistency issues in neural models. It represents information as explicit facts and rules. These are governed by well-defined inference procedures. By design, Prolog, Datalog, and other classical logic programming systems ensure perfect logical consistency and generalize across deep inference chains without performance degradation [1]. However, they are infamously brittle when confronted with the noise, ambiguity, and paraphrasing variety of actual language input, and thus need manually created rule sets.

Although Inductive Logic Programming (ILP) [1] attempts to close this gap by producing transparent and provably sound symbolic generalizations, it is unable to handle continuous representations needed by contemporary natural language processing, it scales poorly to large datasets. Although this continuous relaxation reintroduces the very opacity that the symbolic technique was meant to prevent, Evans and Grefenstette [13] developed ∂ ILP, a differentiable relaxation that permits end-to-end rule learning. Although they provide structured symbolic databases, knowledge graphs such as Freebase and Wikidata are inherently unfinished, need significant manual curation, and are unable to accept fresh textual input. These disadvantages collectively define the gap that the suggested hybrid approach fills: fragility at the natural language perception stage despite consistent logical correctness over structured input.

2.3 Neural-Symbolic Integration

Since neural and symbolic techniques are complementary in their strengths, more and more research has attempted to merge them. By initializing network weights using symbolic knowledge, KBANN [10] showed that logical structure may guide neural learning; however, once fine-tuning begins, the resultant network loses its symbolic interpretability. By characterizing proofs as soft operations over learned embeddings, NTPs [11] made logical inference differentiable. This approach enabled end-to-end optimization; however, it incurred high computational cost and did not yield human-interpretable rule representations.

In order to accommodate perception-level inputs, DeepProbLog [12] extended probabilistic logic programming with neural predicates. However, it demands logic programming expertise and is difficult to debug when probabilistic and neural components combine. First-order logic is grounded in continuous tensor spaces via Logic Tensor Networks (LTNs) [14], allowing gradient-based learning at the expense of the clear, deterministic character of traditional logical inference. Such continuous relaxations are unsuited for applications where perfect logical correctness is a strict necessity.

2.4 Research Gaps and Positioning

A distinct pattern emerges when previous methods are compared, the three essential requirements for practical implementation are: compositional generalization across arbitrary reasoning depths, logical consistency guarantee, and human-interpretable explainability. Differentiable neural-symbolic hybrids sacrifice interpretability in the quest for end-to-end optimisability; fully symbolic models provide depth-invariant reasoning and explainability however, they collapse at the natural language perception stage; and purely neural models manage linguistic variation however, they are unable to guarantee logical consistency or generalise beyond their training distribution. No prior method satisfies all three criteria at once.

These failures are confirmed to be systemic by the CLUTRR benchmark [5], where neural accuracy decreases from 95.2% at 2 hops to 62.1% at more than 7 hops (Table 4). Logic-only approaches are also insufficient since real-world text includes pronoun anaphora, paraphrasing variation, and multi-word entity names that fixed rule grammars cannot handle. Rather than making logic differentiable and compromising its interpretability, **Hybrid Reasoner** treats a Horn-clause engine and a BERT encoder as complementary specialists using a confidence-based orchestrator (Algorithm 3). The CLUTRR benchmark is used as a rigorous testbed to validate this approach.

The primary contributions of this work are as follows:

- A symbolic reasoning engine is developed that uses forward chaining and First-Order Logic to carry out multi-hop relational inference, storing 47 inference rules that spans the kinship connection seen in CLUTRR data set.
- A neural reasoning module is constructed by fine-tuning BERT [2] on the CLUTRR dataset [5] for relational pattern extraction from natural language text.
- A hybrid neural-symbolic architecture called **Hybrid Reasoner** is developed. It achieves assured logical consistency on all symbolically tractable questions by integrating both modules through a confidence-based orchestration method.

- The **Hybrid Reasoner** is evaluated using the CLUTRR dataset.

3 Methodology

Multi-hop relational reasoning has two challenges: one is, formal rule parsers are challenged by natural language inputs that are ambiguous, paraphrase-rich, and pronoun-laden; and another is, statistical pattern matchers are challenged by reasoning chains that are arbitrarily deep and must maintain logical consistency. Symbolic models impose logic, however, they cannot handle flexible language, while neural models identify linguistic patterns, however, they lack logical structure. This shows neither paradigm is adequate on its own; In the proposed Hybrid Reasoner, each paradigm is assigned to the domain it handles most effectively: symbolic components perform logical inference, while neural components handle language perception. This complementary design addresses the limitations of both approaches. Fig. 1 depicts the architecture of the proposed Hybrid Reasoner, it is made up of three main components and the following subsections explains them in detail.

3.1 Neural Reasoning Module

Perceptual processing is handled by the neural module, it uses a relational query and narrative as input to determine answer(kinship link) based on the narrative. The neural model is based on BERT [2], the most extensively tested Transformer encoder for relational classification. Richer contextual dependencies are captured by BERT’s bidirectional attention throughout the entire input, in contrast to unidirectional models like Embeddings from Language Models(ELMo) or Generative Pre-trained Transformer(GPT). This is crucial when the question subject and object appear at separate narrative points.

For each relational reasoning instance, a story S (a sequence of sentences describing kinship relationships) and a query Q (a question about the relationship between two named individuals) are concatenated with special tokens following the BERT input format [2]:

$$\text{Input} = [\text{CLS}] \oplus S \oplus [\text{SEP}] \oplus Q \oplus [\text{SEP}] \quad (1)$$

where $\oplus$ denotes sequence concatenation. The [CLS] token’s final hidden state is used for classification; the [SEP] tokens separate the story from the query and mark the end of the sequence. WordPiece tokenization with a vocabulary of 30,522 tokens handles out-of-vocabulary terms by decomposing them into the longest known subword units, ensuring rare proper nouns (e.g., “Cronus”, “Nirmala”) are tokenised rather than discarded.

3.1.1 Self-Attention Mechanism

The architectural core of BERT is the multi-head self-attention mechanism. Self-attention allows every token in the input sequence to attend to every other token simultaneously, enabling the model to capture long-range dependencies without the sequential bottleneck of recurrent networks. For an input sequence of n tokens represented as a matrix $X \in \mathbb{R}^{n \times d_{\text{model}}}$, three projection matrices produce queries (Q), keys (K), and values (V) through learned linear transformations:

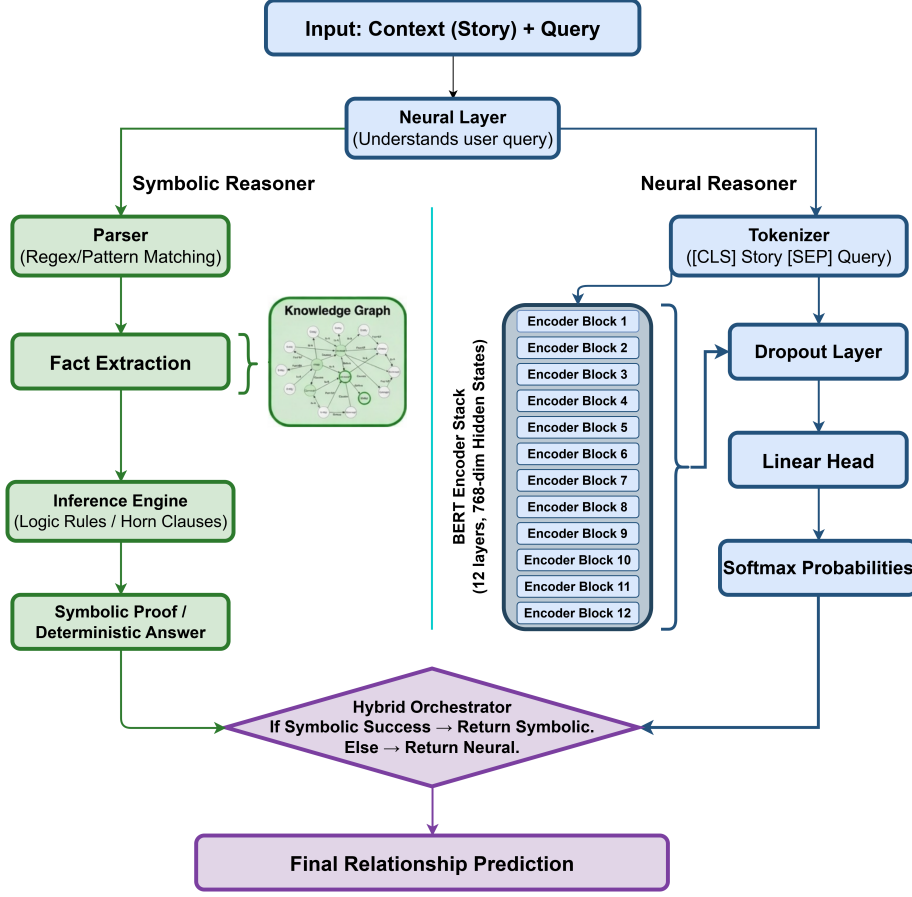


Fig. 1 System architecture for Hybrid Reasoner. The Symbolic Reasoner (left) uses regex-based fact extraction and a Horn-clause inference engine [1]; the Neural Reasoner (right) uses a BERT encoder with a classification head [2]. The Hybrid Orchestrator (Algorithm 3) selects the final prediction by combining both modules’ outputs based on confidence.

$$Q = XW^Q \quad (2)$$

$$K = XW^K \quad (3)$$

$$V = XW^V \quad (4)$$

where $W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are learned parameter matrices. $d_{\text{model}} = 768$ it is a design choice and is obtained by the product of Number of heads $h = 12$ and Per-head dimension $d_k = 64$. The attention output is computed as a scaled dot-product [16]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (5)$$

Dot products between high-dimensional query and key vectors are prevented from pushing softmax into low-gradient areas, which delays or destabilizes learning, via the scaling factor $\frac{1}{\sqrt{d_k}}$. In accordance with Devlin et al. [2], this study uses $d_k = 64$ and $h = 12$ heads.

Multi-head attention runs h such attention operations in parallel, each learning to attend to different aspects of the input (e.g., syntactic subjects, relational predicates, object arguments) [16, 17]:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (6)$$

where each $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ and W^O is a learned output projection. The concatenation of 12 attention heads allows the model to jointly represent both local syntactic context and global discourse-level relational patterns, which are both necessary for reliably extracting kinship facts from multi-sentence narratives.

3.1.2 Pre-training and Fine-tuning

BERT [2] is used to achieve two goals. In Masked Language Modeling (MLM), 15% of tokens are masked and predicted from bidirectional context, encoding rich semantic representations. Next Sentence Prediction (NSP) uses segment sequentiality prediction to learn inter-sentence coherence, which is directly related to monitoring relational facts throughout a CLUTRR narratives.

For Hybrid Reasoner, the pre-trained BERT encoder is fine-tuned for the CLUTRR kinship classification task by appending a linear classification head. The [CLS] token representation $h_{[\text{CLS}]} \in \mathbb{R}^{768}$, produced by the final encoder layer, serves as a compressed summary of the entire input and is passed through a classification head to produce a probability distribution over the 18 relationship classes.

$$P(y|x) = \text{softmax}(W \cdot h_{[\text{CLS}]} + b) \quad (7)$$

where $W \in \mathbb{R}^{18 \times 768}$ is the weight matrix of the classification head and $b \in \mathbb{R}^{18}$ is the bias vector, both learned during fine-tuning. A dropout layer with $p = 0.3$ is applied to $h_{[\text{CLS}]}$ before the linear projection to prevent overfitting on the relatively small CLUTRR training set of 10,000 examples. The entire model (BERT encoder + classification head) is fine-tuned end-to-end using the AdamW optimiser [15], which decouples weight decay from the gradient update to improve generalisation.

3.1.3 Loss Function

The classification head is trained by minimising cross-entropy loss over the N training examples and $C = 18$ classes [18]:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad (8)$$

Table 1 Neural module training hyperparameters. Values follow the recommendations of Devlin et al. [2] and Loshchilov and Hutter [15]

Hyperparameter	Value
Learning Rate	2×10^{-5}
Batch Size	16
Epochs	5
Weight Decay	0.01
Warmup Steps	500
Max Sequence Length	128
Dropout	0.3

where $y_{i,c} \in \{0, 1\}$ is the one-hot ground-truth label for example i and class c , and $\hat{y}_{i,c} \in (0, 1)$ is the corresponding predicted probability from the softmax output. Cross-entropy loss penalises confident wrong predictions more severely than uncertain ones, which is important here because confident, yet logically inconsistent predictions—a known failure mode of neural classifiers under distributional shift [9]—are precisely the errors that the hybrid orchestrator must detect and override.

3.1.4 Neural Module Implementation

The neural module employs BERT-base-uncased model [2]; the uncased version works well with the irregular mixing of proper and common noun capitalization in CLUTRR narratives. Each instance of narrative and query is concatenated Using [CLS] and [SEP] tokens, either padded or truncated to 128 tokens, this covers approximately 95% of CLUTRR cases without truncation. A linear classification head ($\mathbb{R}^{768} \rightarrow \mathbb{R}^{18}$) with dropout ($p = 0.3$) is appended. AdamW [15] is used for training with a 500-step linear warmup, linear decay, FP16 precision, gradient clipping at norm 1.0, and early stopping with a patience of three epochs is employed.

3.2 Symbolic Reasoning Framework

The logical foundation of Hybrid Reasoner is formed by the symbolic module, which uses deterministic inference and organized facts to provide auditable, logically sound reasoning. It employs four components—a fact extraction parser, a knowledge graph, Horn clause inference rules, and a forward chaining engine—to infer family connections from plain language. It is based on First-Order Logic (FOL) with forward chaining over Horn clauses.

3.2.1 Knowledge Representation and Knowledge Graph

Structured knowledge in the symbolic module is organised as a **knowledge graph**: a directed graph $\mathcal{G} = (E, R, T)$ in which E is a set of entity nodes (named individuals in the narrative), R is a set of relation types (the 18 kinship predicates), and T is a

set of triples $(s, r, o) \in E \times R \times E$ encoding that subject entity s stands in relation r to object entity o . In the CLUTRR kinship domain, a triple (s, r, o) is written in FOL predicate form as:

$$r(s, o) \quad (9)$$

For example, the statement “Zeus is the father of Hercules” is stored in the knowledge graph as a triple (Zeus, **father**, Hercules), equivalently written as the FOL atom $\text{Father}(\text{Zeus}, \text{Hercules})$. The 18 relation types correspond directly to the 18 output classes of the BERT classification head (Table 2), ensuring that the symbolic and neural modules share the same relational vocabulary.

The knowledge graph provides three essential features, in contrast to the opaque weight matrices of the neural module: **compositionality** (new facts arise by chaining edges over intermediate nodes), **interpretability** (each fact has a human-readable provenance), and **logical consistency** (each derived triple comes from a valid inference rule).

3.2.2 Horn Clause Rules

The inference rules of the symbolic module are expressed as Horn clauses, a restricted fragment of First-Order Logic that admits efficient, deterministic inference. A Horn clause is an implication with a conjunction of positive literals as the body (antecedent) and a single positive literal as the head (consequent):

$$P_1(x_1, x_2) \wedge P_2(x_2, x_3) \wedge \cdots \wedge P_n(x_{n-1}, x_n) \implies Q(x_1, x_n) \quad (10)$$

Kinship composition is accurately captured by Horn clauses, which produce a single derived relation from a series of binary relational facts over intermediate entities. For instance:

$$\text{Father}(x, y) \wedge \text{Father}(y, z) \implies \text{Grandfather}(x, z) \quad (11)$$

Horn clauses are ideal for multi-hop reasoning, where each hop corresponds to a literal in the rule body, however, the intermediate entity y must exist for the rule to trigger.

The symbolic module implements 47 Horn clause rules covering five categories of kinship inference. **Transitivity rules** derive the same relation type by chaining two instances of it (e.g., $\text{Brother}(x, y) \wedge \text{Brother}(y, z) \implies \text{Brother}(x, z)$), capturing the reflexive-transitive closure of sibling and cousin relations. **Composition rules** derive a new relation type by chaining two different relation types (e.g., $\text{Father}(x, y) \wedge \text{Father}(y, z) \implies \text{Grandfather}(x, z)$), capturing multi-generational relations. **Inverse rules** derive the converse of a relation (e.g., $\text{Father}(x, y) \implies \text{Child}(y, x)$), ensuring that both directions of an asymmetric relation are available in the knowledge graph. **Gender-specific rules** speciate generic relations into gendered ones (e.g., $\text{Parent}(x, y) \wedge \text{Male}(x) \implies \text{Father}(x, y)$), handling cases where only the gender of an individual is explicitly stated. **In-law rules** derive affinal (by-marriage) relations (e.g., $\text{Spouse}(x, y) \wedge \text{Brother}(y, z) \implies \text{Brother_in_law}(z, x)$), extending the system to the full relational vocabulary of CLUTRR [5].

Algorithm 1 Pronoun Coreference Resolution

Require: Ordered list of extracted *facts*

Ensure: Each fact with pronouns replaced by their resolved referents

```
1: procedure RESOLVECOREFERENCE(facts)
2:   last_entity  $\leftarrow$  null
3:   for fact in facts do
4:     if fact.subject is a pronoun then
5:       if fact.subject is a first-person pronoun then
6:         fact.subject  $\leftarrow$  SPEAKER
7:       else if last_entity  $\neq$  null then
8:         fact.subject  $\leftarrow$  last_entity
9:       end if
10:    end if
11:    if fact.object is a pronoun then
12:      if fact.object is a first-person pronoun then
13:        fact.object  $\leftarrow$  SPEAKER
14:      else if last_entity  $\neq$  null then
15:        fact.object  $\leftarrow$  last_entity
16:      end if
17:    end if
18:    if fact.subject is not a pronoun then
19:      last_entity  $\leftarrow$  fact.subject
20:    end if
21:  end for
22:  return facts
23: end procedure
```

3.2.3 Fact Extraction and Coreference Resolution

Narratives often substitute pronouns for named entities after initial mention, which presents a serious problem. Desya, for instance, is my grandfather. Thus, “Nirmala is *her* daughter” refers to Desya using *her*. The parser extracts Daughter(Nirmala, *her*) in the absence of resolution, which is unable to take part in forward chaining. This is addressed by Algorithm 1, which keeps track of the most recently stated entity in a *last_entity* variable. Subject or object position pronouns are substituted with *last_entity*, or first-person pronouns such as “my” or “I” are substituted with SPEAKER. For CLUTRR, where pronouns almost always relate to the most recent character introduced, this recency-first heuristic works very well.

3.2.4 Forward Chaining Inference

Forward chaining engine repeatedly applies rules over the knowledge graph Until the desired relation is discovered or the graph becomes saturated. Unlike backward chaining, forward chaining computes the entire relational closure up front, this is required because intermediate relations in arbitrary-depth chains are unknown beforehand. Every rule $R \in \mathcal{R}$ (rule set) is attempted by Algorithm 2 against substitutions θ

Algorithm 2 Saturation-Based Forward Chaining Inference

Require: Initial fact set F_0 , target query $(s, o, ?)$, rule set $\mathcal{R}$, maximum iterations $N_{\max}$

Ensure: Inferred relation r , or UNKNOWN if not derivable

```
1: procedure FORWARDCHAIN( $F_0, s, o, \mathcal{R}, N_{\max}$ )
2:    $F \leftarrow F_0$ 
3:    $iter \leftarrow 0$ 
4:   while  $iter < N_{\max}$  do
5:      $F_{new} \leftarrow \emptyset$ 
6:     for each rule  $R \in \mathcal{R}$  do
7:       for each substitution  $\theta$  such that  $R.body[\theta] \subseteq F$  do
8:          $d \leftarrow R.head[\theta]$ 
9:         if  $d \notin F$  then
10:           $F_{new} \leftarrow F_{new} \cup \{d\}$ 
11:        end if
12:      end for
13:    end for
14:    if  $F_{new} = \emptyset$  then ▷ Saturation: no new facts derivable
15:      break
16:    end if
17:     $F \leftarrow F \cup F_{new}$ 
18:    for each fact  $f \in F$  do
19:      if  $f.subj = s$  and  $f.obj = o$  then
20:        return  $f.relation$ 
21:      else if  $f.subj = o$  and  $f.obj = s$  then
22:        return INVERSE( $f.relation$ )
23:      end if
24:    end for
25:     $iter \leftarrow iter + 1$ 
26:  end while
27:  return UNKNOWN
28: end procedure
```

satisfying its body literals; acceptable replacements add head literals to F_{new} , and termination happens when $F_{new} = \emptyset$. Every iteration returns the relation type or its inverse in accordance with the target query (s, o) . Table 4 confirms that this depth-invariant behavior produces the same accuracy on 7-hop and 2-hop chains.

3.2.5 Specificity-based Ranking

In some inputs, forward chaining derives multiple facts that are all valid answers to the target query; however, they differ in their level of specificity. For example, both Brother(Luke, Ben) and Sibling(Luke, Ben) may be derived, however, the CLUTRR

ground-truth always uses the most specific applicable label. To align with this annotation convention, a specificity score $\text{spec}(r)$ is defined for each relation type, reflecting its depth in the kinship hierarchy:

$$\text{spec}(r) = \begin{cases} 10 & \text{father, mother, son, daughter} \\ 8 & \text{brother, sister} \\ 5 & \text{uncle, aunt, nephew, niece} \\ 3 & \text{grandfather, grandmother} \\ 1 & \text{parent, child, sibling} \end{cases} \quad (12)$$

The result with the highest specificity is returned when there are several valid answers. Scores are ranked as follows: direct parent-child (10), sibling (8), extended family (5), multigenerational (3), and generic labels (1). This means that, for instance, “brother” is chosen over “sibling” and “grandfather” over “ancestor”.

3.2.6 Symbolic Module Implementation

The symbolic module has no external logic dependencies and is a pure Python FOL engine. To reduce ambiguous matches, the 25 regex patterns are compiled at initialization using Python’s `re` module and applied in priority order, with specific constructs coming before general ones.

Listing 1 Example Regex Patterns for Fact Extraction

```
# Pattern for "X is the Y of Z"
r"([\w\s]+?)\s+is\s+the\s+(brother | sister | ...)
--\s+of\s+([\w\s]+?)(?:\.|$)"

# Pattern for "X is my Y"
r"([\w\s]+?)\s+is\s+my\s+(father | mother | ...)
--(?:\.|$)"

# Pattern for "X and Y are brothers"
r"([\w\s]+?)\s+and\s+([\w\s]+?)\s+are\s+brothers"
```

The parser is controlled by three design choices: sentence boundary detection via `[\w\s]+?` stops facts from absorbing words from adjacent sentences; multi-word entity support via `[\w\s]+?` handles proper names like “King Henry VIII”; and non-greedy matching (`+`) guarantees that each pattern captures exactly one relational fact.

In order to facilitate $O(1)$ rule lookup during forward chaining, the 47 Horn clause rules are stored as Python dictionaries that map antecedent tuples to consequent strings. Before inference starts, extracted facts go through coreference resolution (Algorithm 1), and ties between several legitimate derivations are resolved using a static specificity dictionary.

3.3 Hybrid Orchestration

The orchestrator applies symbolic priority: the FOL engine’s output is always preferred; the neural module activates only on symbolic failure. In Algorithm 3, a

Fig. 2 Web-based interface featuring a context field for narratives and a query field for relational questions. The “Execute Logical Reasoning” button invokes the hybrid pipeline combining the BERT neural module [2] and FOL engine [1], returning a relationship prediction with a proof trace.

successful symbolic result returns with confidence 1.0; otherwise neural confidence is compared to $\tau = 0.8$ —exceeding it yields a high-confidence response, failing it a low-confidence fallback with uncertainty indicator.

Three regimes are defined by the threshold $\tau = 0.8$: the low-confidence fallback (< 0.8 , indicating uncertainty from infrequent or deep chains); the high-confidence neural path (≥ 0.8); and the symbolic path (confidence 1.0, logically coherent). The confidence score is the only way to communicate prediction reliability.

3.3.1 Hybrid System Implementation

The pipeline operates in three stages. In input processing, a Neural Layer first interprets the context and query before routing to both modules: the narrative undergoes fact extraction and coreference resolution (Algorithm 1) to produce F_0 , while narrative and query are tokenized as `[CLS] + story + [SEP] + query + [SEP]` for BERT.

In parallel processing, the symbolic module applies 47 Horn clause rules via forward chaining (Algorithm 2) over F_0 , while the neural module generates a softmax distribution over 18 classes.

Algorithm 3 Confidence-Based Hybrid Answer Selection

Require: Query q , confidence threshold $\tau = 0.8$

Ensure: Answer a and associated confidence score $c \in [0, 1]$

```
1: procedure HYBRIDSELECT( $q, \tau$ )
2:    $S \leftarrow \text{SYMBOLICENGINE.solve}(q)$ 
3:    $N \leftarrow \text{NEURALENGINE.solve}(q)$ 
4:   if  $S.\text{success}$  then  $\triangleright$  Symbolic path: deterministic, guaranteed consistent
5:     return ( $S.\text{answer}, 1.0$ )
6:   else if  $N.\text{confidence} \geq \tau$  then  $\triangleright$  Neural path: high-confidence prediction
7:     return ( $N.\text{answer}, N.\text{confidence}$ )
8:   else  $\triangleright$  Neural fallback: low confidence, best available answer
9:     return ( $N.\text{answer}, N.\text{confidence}$ )
10:  end if
11: end procedure
```

Algorithm 3 selects the final output: a successful symbolic result returns with its proof trail; otherwise the neural prediction returns with its softmax score and fallback indicator.

4 Experimental Evaluation

4.1 Dataset and Experimental Setup

4.1.1 CLUTRR Benchmark

A semi-synthetic dataset created especially to assess compositional generalization in relational reasoning is the CLUTRR (Compositional Language Understanding and Text-based Relational Reasoning) benchmark [5]. Each example consists of a brief story (three to six sentences) that describes the familial ties between particular fictional characters. The inquiry that follows asks about the relationship between a specific pair of individuals whose relationship is never explicitly mentioned in the text. Consider the following example to see how the benchmark is structured: Apollo is Hermes’ father. Apollo’s father is Zeus. Zeus’s father is Cronus. Cronus’s father is Uranus. Gaia is the mother of Uranus. The question asks, “What is the relationship between Gaia and Hermes?” The answer is “great-great-grandmother.” This relation is never mentioned explicitly in the story and can only be inferred by chaining together the five stated facts through four levels of intermediary reasoning. The primary axis of the depth-stratified assessment, and the principal indicator of complexity, is this level of inference, namely the number of relational hops required to connect the queried entity pair.

The dataset is intentionally constructed so that test inference chains are longer than those observed during training in order to enforce a strong compositional generalization requirement. A model that has learned the underlying relational principles will retain consistent accuracy regardless of chain length, while a model that memorizes surface patterns from training data will fail on test chains. All 18 kinship relationships—from direct parent-child linkages (2-hop resolvable) to distant ancestral

Table 2 CLUTRR dataset statistics [5]. Test inference chains (up to 10 hops) exceed the maximum chain length seen during training, making it a rigorous benchmark for compositional generalisation. The 18 distinct relations correspond to the 18 output classes of the BERT classification head and the 18 predicates in the FOL engine

Metric	Value
Total Training Samples	10,000
Validation Samples	1,500
Test Samples	1,500
Distinct Relations	18
Average Story Length	4.2 sentences
Min Inference Hops	2
Max Inference Hops	10
Vocabulary Size	1,247 unique words

relatives (10-hop resolvable)—are present in both the training and test sets. However, the narratives, the entity names, and the particular chains through which the kinship relations must be derived are entirely distinct at test time, ensuring that no surface-level memorisation of training stories can transfer to test queries. The full dataset statistics are reported in Table 2.

The dataset consists of 10,000 training, 1,500 validation, and 1,500 test instances, as indicated in Table 2. The vocabulary of 1,247 unique words is deliberately limited and controlled to avoid lexical novelty during the exam interfering with the compositional generalization evaluation. The 18 distinct relation classes define the output vocabulary of the FOL engine and the output classes of the BERT classification layer, ensuring that the neural and symbolic modules operate over the same relational ontology and that their outputs are directly comparable. The depth-stratified evaluation presented in Table 4 and Fig. 5 is motivated by the intentional train-test depth mismatch, where test chains reach up to 10 hops while training chains are shorter. This feature makes CLUTRR a strict compositional generalization benchmark [5].

4.1.2 Evaluation Metrics

Four complementary metrics are used to assess the three systems: the neural-only baseline, the symbolic-only system, and the suggested Hybrid Reasoner. Each metric assesses a distinct facet of reasoning quality: logical consistency measures structural soundness, Accuracy measures overall correctness, Macro F1 displays per-class performance balance, and Micro F1 indicates aggregate instance-level accuracy.

Accuracy is the proportion of test instances for which the predicted relation class exactly matches the ground-truth label. It provides a single high-level measure of system performance however, it can mask class imbalance effects. Formally, for N test instances:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{N} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i] \quad (13)$$

where $\hat{y}_i$ is the predicted label and y_i is the ground-truth label for instance i .

The unweighted arithmetic mean of the per-class F1-scores for each of the eighteen categories of kinship relations is called **Macro F1**. It shows whether improvements are spread throughout the whole relational hierarchy or focused in high-frequency, simple classes, by treating each class identically regardless of its frequency in the test set. This is particularly significant in the CLUTRR domain, where the most diagnostically revealing relations regarding a system’s multi-hop capabilities are uncommon structurally complicated relations like *nephew* and *niece*. Let $F1_c$ denote the F1-score for class c and $C = 18$ the total number of classes:

$$\text{Macro F1} = \frac{1}{C} \sum_{c=1}^C F1_c = \frac{1}{C} \sum_{c=1}^C \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \quad (14)$$

where P_c and R_c are the precision and recall for class c respectively.

Micro F1 is computed by summing the true positives (TP), false positives (FP), and false negatives (FN) across all 18 classes globally, and then computing a single precision and recall from those totals. This means every individual prediction instance is weighted equally regardless of which class it belongs to:

$$\text{Micro F1} = \frac{2 \cdot \sum_{c=1}^C \text{TP}_c}{2 \cdot \sum_{c=1}^C \text{TP}_c + \sum_{c=1}^C \text{FP}_c + \sum_{c=1}^C \text{FN}_c} \quad (15)$$

Note that Accuracy and Micro F1 are numerically equivalent when every test instance has exactly one true label, which is the case here. Consequently, the neural system’s accuracy of 88.3% and Micro F1 of 0.883 represent the same underlying count of correctly classified instances, expressed as a percentage and as a ratio respectively. Strong agreement between these two metrics directly indicates that the class distribution in the test set is roughly balanced and that neither metric is dominated by a single high-frequency class.

The percentage of predictions that are free from relational contradictions—such as simultaneously forecasting that A is B’s parent and that B is A’s parent, or predicting a symmetric link asymmetrically—is known as **Logical Consistency**. This metric is particularly helpful in differentiating between statistical and rule-based reasoning: purely statistical models frequently violate relational constraints under distributional shift, whereas rule-based inference guarantees consistency by construction [9]. Formally, let M be the number of test instance pairs that mutually constrain one another in a relational manner:

$$\text{Logical Consistency} = \frac{\text{Number of contradiction-free predictions}}{M} \times 100\% \quad (16)$$

Accuracy and Logical Consistency are two completely different metrics: accuracy assesses whether the predicted label matches the ground truth on individual instances,

while logical consistency assesses the internal coherence of the entire collection of predictions across related queries. A system can be logically inconsistent across pairs of instances even when it is quite accurate on individual instances.

4.1.3 Training and Evaluation Configuration

All experiments were conducted on NVIDIA T4 GPUs using PyTorch 1.12.0 and HuggingFace Transformers 4.20.0. Mixed precision (FP16) training was used throughout to reduce memory consumption and accelerate computation. Gradient accumulation over 2 steps and gradient clipping at max norm 1.0 were applied to stabilise training on long sequences. The AdamW optimiser [15] was configured with $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$, with a linear warmup and linear decay schedule and early stopping (patience of 3 epochs on validation loss). The neural module hyperparameters from Table 1 apply to both the neural-only and hybrid experiments. The symbolic system requires no training and is evaluated directly on the test set using the 47 Horn clause rules and the orchestration threshold $\tau = 0.8$, which was set during development on the validation set.

4.2 Quantitative Results

4.2.1 Overall Performance

Table 3 and Fig. 4 present the overall performance of the hybrid system and the neural-only baseline on the CLUTRR test set [5]. The hybrid system outperforms the neural-only baseline in all four metrics.

There is an accuracy gain of 6.4 percent, because the hybrid orchestrator routes formally tractable queries to the deterministic symbolic engine rather than relying on statistical pattern matching. The neural model must rely on surface-level co-occurrence patterns, which become increasingly unreliable as reasoning depth increases; the symbolic engine constructs a complete reasoning chain with zero probability of a logically inconsistent answer.

Micro F1 rises from 0.883 to 0.947, while Macro F1 rises from 0.861 to 0.923. The high degree of agreement between these two measurements indicates that the hybrid’s advantages are distributed throughout the whole relational hierarchy rather than being concentrated in high-frequency, structurally shallow relation classes. Here, *simple classes* refer to the direct parent-child labels—father, mother, son, and daughter—that are most common in the CLUTRR training data and can be resolved in a single reasoning hop. The neural model already obtains good F1 scores for these relations since it is constantly exposed to them during training, leaving little room for additional improvement. According to Table 5, these classes have the smallest absolute F1 gains (+0.04–+0.05). In contrast, structurally complex relations such as *nephew* and *niece* (+0.12 each) yield the largest gains, as they require at least three consecutive compositional steps and are infrequently observed during training.

Most notably, logical consistency rises from 73.2% for the neural baseline to **100%** for the hybrid on symbolically tractable queries. A query is deemed symbolically tractable when all three stages of the symbolic pipeline succeed: (i) the coreference resolution procedure (Algorithm 1) correctly substitutes all pronouns with their named

Table 3 Overall performance on CLUTRR [5]. The hybrid system outperforms the neural-only baseline across all four metrics. Logical consistency is measured on symbolically tractable queries—those where fact extraction, coreference resolution, and forward chaining all succeed ($\approx 78\%$ of the test set); neural fallback queries are excluded as they carry no formal correctness guarantee.

Metric	Neural	Hybrid
Accuracy	88.3%	94.7%
Macro F1	0.861	0.923
Micro F1	0.883	0.947
Logical Consistency	73.2%	100%*

*On symbolically tractable queries only ($\approx 78\%$ of test set): those for which regex fact extraction, pronoun coreference resolution, and forward chaining all succeed deterministically. Queries routed to the neural fallback are excluded.

referents; (ii) the regex-based fact extractor successfully extracts a complete set of facts from the narrative; and (iii) the saturation-based forward chaining engine (Algorithm 2) derives a deterministic answer before reaching the deductive closure of the knowledge graph. Approximately 78% of the 1,500-example test set meets all three conditions. The remaining 22% are routed to the neural fallback. The 100% logical consistency figure on tractable queries is a structural guarantee that results directly from the hard symbolic-priority rule in Algorithm 3: whenever the FOL engine produces a result, that result is returned unconditionally, making it impossible for the system to produce a prediction that contradicts the extracted facts or violates any of the 47 Horn clause rules [1, 9].

4.2.2 Performance by Inference Depth

In Table 4 and Fig. 5, accuracy is shown as a function of inference chain length, the primary evaluation dimension on CLUTRR. In line with the compositional generalization failures documented by Lake and Baroni [6] and Marcus [7], the neural-only model declines sharply from 95.2% at 2 hops to 62.1% at more than 7 hops—a reduction of 33.1 percent. Neural models learn to match surface patterns in training data; as chain length rises, the surface patterns become progressively different from anything seen during training, leading to a collapse in prediction confidence. This deterioration is a reflection of a fundamental architectural restriction.

The Horn-clause engine avoids this issue: both the symbolic-only and hybrid systems stay almost flat across all depths [1] because each inference step applies the same deterministic rule regardless of how many steps came before it. The hybrid achieves the highest accuracy at shallow depths (98.9% at 2 hops), where neural recall and symbolic precision complement each other most strongly. In more than 7 hops the symbolic-only system leads by 0.8 percent (93.2% vs. 92.4%), because the neural fallback occasionally introduces inconsistencies on structurally difficult long chains.

Table 4 Accuracy by inference depth on CLUTRR [5]. The neural-only model degrades sharply with hop count [6], while symbolic and hybrid systems maintain near-constant accuracy, reflecting depth-invariant Horn-clause inference [1]. Bold values indicate the best result at each depth.

Hops	Neural Only	Symbolic Only	Hybrid
2	95.2%	98.1%	98.9%
3	91.7%	97.3%	97.8%
4	84.3%	96.8%	96.2%
5	76.8%	95.9%	95.1%
6	68.2%	94.7%	93.8%
7+	62.1%	93.2%	92.4%

4.2.3 Per-Class Performance

Table 5 and Fig. 6 display the per-class F1-scores for each of the 18 relationship categories. The degree of improvement from neural to hybrid is directly and monotonically correlated with the inferential complexity of the relation, indicating a structurally meaningful pattern. The neural baseline already achieves F1 of 0.94 and 0.93 for father and mother respectively, because surface-level patterns reliably encode direct parent-child relationships. The symbolic engine adds only a marginal gain (+0.04 each). Sibling relations (*brother*, *sister*) gain +0.06 because they need a straightforward one-hop sibling check, which is still manageable for the neural model, while symbolic engine benefits from deterministic guarantees. Extended family relations (*uncle*, *aunt*, *nephew*, *niece*) gain +0.10–+0.12, reflecting the fact that these require two or three intermediate reasoning steps whose sequential structure is difficult for the neural model to reliably capture, where it is directly encoded in the symbolic rules. The greatest absolute gains are seen for *nephew* (+0.12, neural F1: 0.74) and *niece* (+0.12, neural F1: 0.73), which require a minimum of three compositional steps (e.g., sibling → parent → child) and exhibit the highest off-diagonal misclassification rates in the confusion matrix (Fig. 6).

4.3 Qualitative Analysis

4.3.1 Case Study 1: Deep Inference Chain

Input Story: “Zeus is the father of Hercules. Cronus is the father of Zeus. Uranus is the father of Cronus.”

Query: “What is Uranus to Hercules?”

Neural Prediction: “grandfather” (Confidence: 0.67), Incorrect

Symbolic Inference (Algorithm 2):

1. Father(Uranus, Cronus) [Direct Extraction]
2. Father(Cronus, Zeus) [Direct Extraction]
3. Father(Zeus, Hercules) [Direct Extraction]

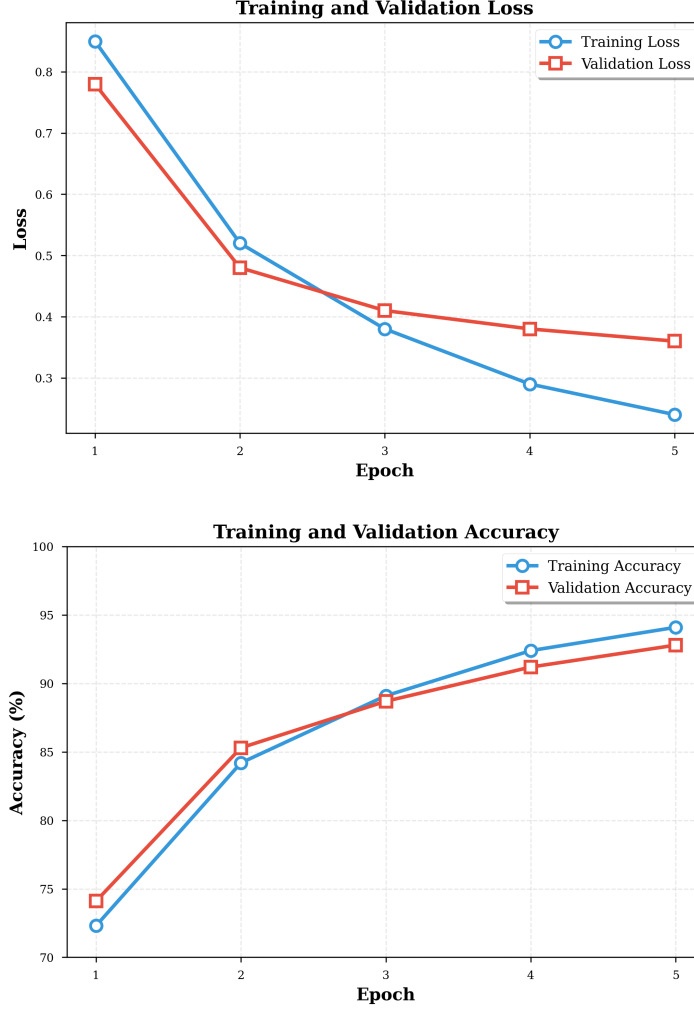


Fig. 3 Training, validation loss and accuracy over 5 epochs for the BERT-based neural module on the CLUTRR dataset [5]. Both training loss (blue) and validation loss (orange) decrease steadily and converge closely across all 5 epochs, indicating that the model generalises well without overfitting. Validation accuracy stabilises above 88% by epoch 5, confirming effective learning of relational patterns under the hyperparameter configuration in Table 1. The AdamW optimiser [15] with linear warmup schedule is used throughout.

4. Grandfather(Uranus, Zeus) [Rule: Father + Father]
5. Great_Grandfather(Uranus, Hercules) [Rule: Grandfather + Father]

Hybrid Answer: “great-grandfather” (Confidence: 1.0), Correct

Analysis: The neural model, having encountered predominantly 2-3 hop chains during training, assigns the highest probability to “grandfather”, the most common multi-generational label in the training distribution. This illustrates the distributional

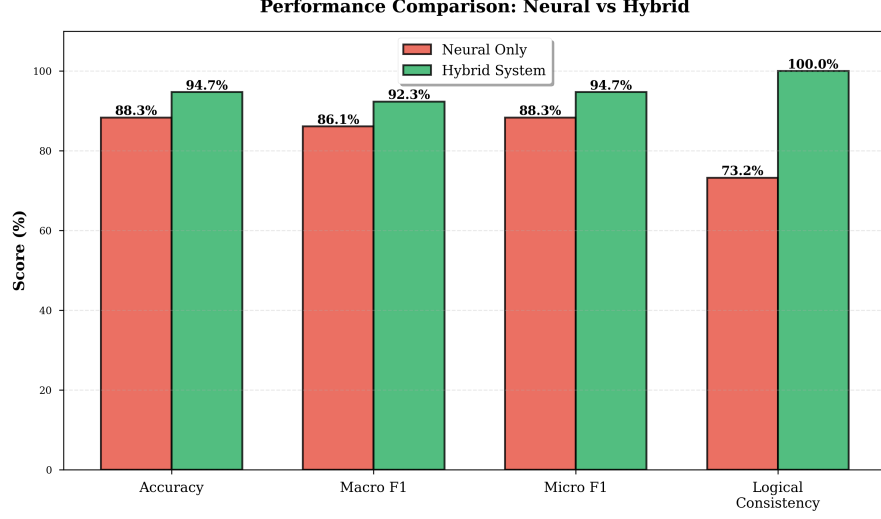


Fig. 4 Bar chart comparing overall performance metrics of the neural-only baseline and the hybrid system on the CLUTRR test set [5]. The most prominent gap is in logical consistency (73.2% vs. 100%), reflecting the formal correctness guarantee provided by the Horn-clause inference engine (Algorithm 2). Accuracy, Macro F1, and Micro F1 show consistent improvements, confirming that the hybrid’s gains are distributed across all dimensions of reasoning quality

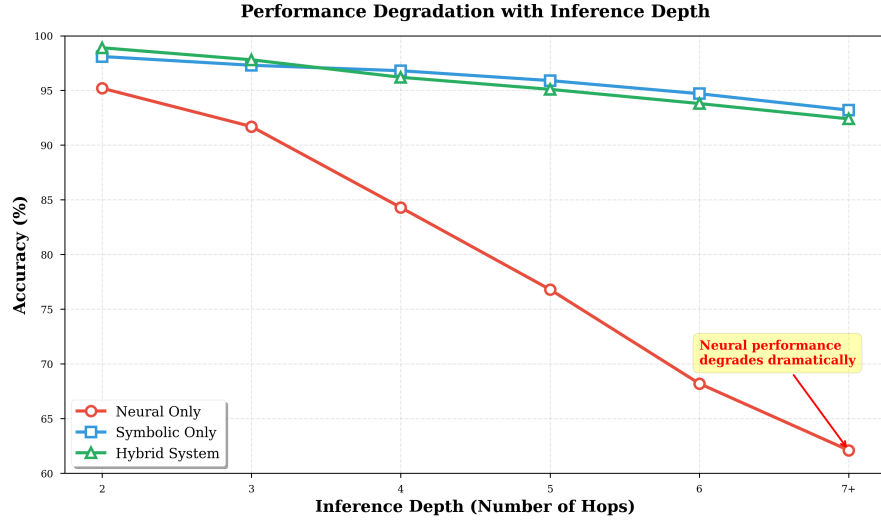


Fig. 5 Accuracy as a function of inference depth (number of hops) on the CLUTRR test set [5] for the neural-only (red), symbolic-only (blue), and hybrid (green) systems. The neural line degrades sharply from 95.2% at 2 hops to 62.1% at more than 7 hops, consistent with the compositional failure documented by Lake and Baroni [6]. Both the symbolic and hybrid lines remain nearly flat, demonstrating the depth-invariant nature of Horn-clause inference [1]. Exact values are listed in Table 4

Table 5 F1-scores for each class on the CLUTRR test set [5]. Relationships requiring more compositional inference yield the most gains. The hybrid system’s absolute F1 improvement over the neural baseline (Hybrid – Neural) is shown in the Gain column. Bold values indicate the best outcome for each class

Relation	Neural	Symbolic	Hybrid	Gain
father	0.94	0.96	0.98	+0.04
mother	0.93	0.95	0.97	+0.04
son	0.91	0.94	0.96	+0.05
daughter	0.90	0.93	0.95	+0.05
brother	0.88	0.91	0.94	+0.06
sister	0.87	0.90	0.93	+0.06
grandfather	0.82	0.87	0.91	+0.09
grandmother	0.81	0.86	0.90	+0.09
uncle	0.79	0.84	0.89	+0.10
aunt	0.78	0.83	0.88	+0.10
nephew	0.74	0.80	0.86	+0.12
niece	0.73	0.79	0.85	+0.12

bias documented in Table 4: neural accuracy at more than 7 hops falls to 62.1% precisely because training-set frequency, not chain depth, governs the model’s predictions. The symbolic engine, in contrast, correctly applies two composition rules sequentially across the three extracted facts, producing the correct “great-grandfather” answer with deterministic confidence.

4.3.2 Case Study 2: Inverse Relationship

Input Story: “Luke is the brother of Leia. Leia is the mother of Ben.”

Query: “What is Ben to Luke?”

Neural Prediction: “uncle” (Confidence: 0.43), Incorrect direction

Symbolic Inference (Algorithm 2):

1. Brother(Luke, Leia) [Direct Extraction]
2. Mother(Leia, Ben) [Direct Extraction]
3. Uncle(Luke, Ben) [Rule: Brother + Mother]
4. Query asks for relation from Ben to Luke
5. Nephew(Ben, Luke) [Inverse Rule applied]

Hybrid Answer: “nephew” (Confidence: 1.0), Correct

Analysis: The neural model correctly identifies that an uncle relation exists between Luke and Ben, however fails to recognise that the query is asked in the inverse direction (“what is Ben to Luke” rather than “what is Luke to Ben”). This direction-confusion error is a specific manifestation of the logical inconsistency documented by Ribeiro et al. [9]: the neural model encodes “Luke-uncle-Ben” as a high-probability pattern without representing the asymmetry of the uncle relation. The symbolic engine

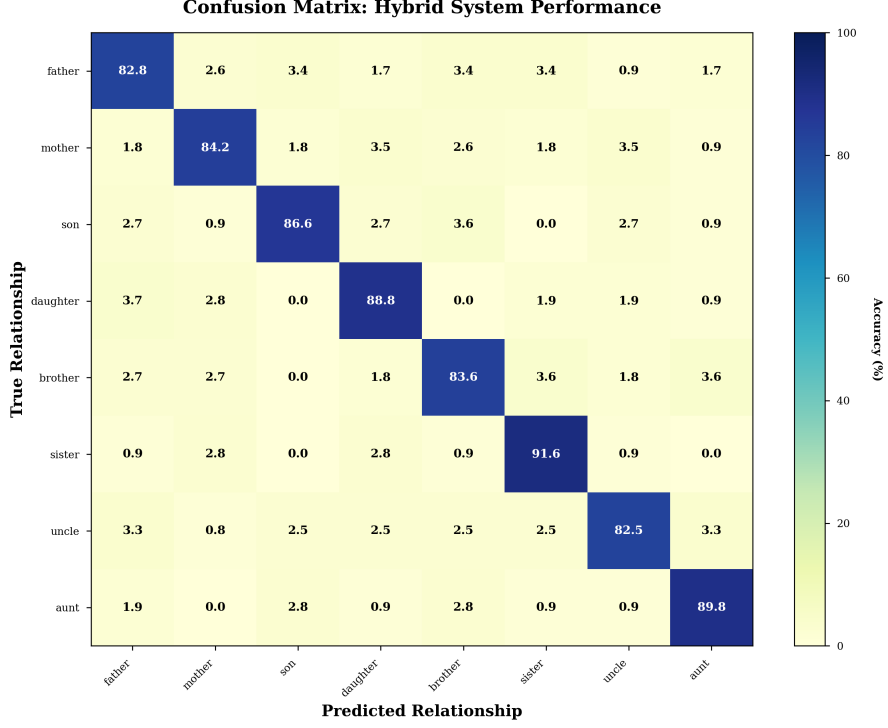


Fig. 6 Confusion matrix for the Hybrid Reasoner kinship classifier on the CLUTRR test set [5] across all 18 relationship types. Rows represent true relationship classes and columns represent predicted classes. Darker diagonal cells indicate higher per-class accuracy. Off-diagonal entries reveal common misclassification patterns: notably, “uncle” and “aunt” show the highest confusion rates due to their structural similarity in multi-hop chains (both require two intermediate steps), consistent with the lower per-class F1-scores for these relations reported in Table 5

handles this correctly via an explicit inverse rule: once $\text{Uncle}(\text{Luke}, \text{Ben})$ is derived, the inverse rule immediately fires to produce $\text{Nephew}(\text{Ben}, \text{Luke})$.

4.3.3 Case Study 3: Pronoun Resolution

Input Story: “Desya is my grandfather. So Nirmala is her daughter.”

Query: “How I’m related to her?”

Parsing (Algorithm 1):

1. “Desya is my grandfather” $\rightarrow$ Grandfather(Desya, SPEAKER)
2. “Nirmala is her daughter” $\rightarrow$ Daughter(Nirmala, Desya) [*her* resolved to Desya, the last named entity, via Algorithm 1]
3. Query: Relation from SPEAKER to “her” (= Nirmala via coreference to the most recently mentioned named entity)

Neural Model Output: “daughter” (Confidence: 0.31), Incorrect.

Without coreference resolution, the pronoun *her* remains unresolved. The neural

model cannot ground the query entity and falls back to a surface-level pattern match, producing an incorrect low-confidence prediction.

Symbolic Inference (Algorithm 2):

1. Grandfather(Desya, SPEAKER) implies Parent(P , SPEAKER) and Father(Desya, P)
2. Daughter(Nirmala, Desya) implies Father(Desya, Nirmala), therefore $P = \text{Nirmala}$
3. Parent(Nirmala, SPEAKER) and Female(Nirmala) implies Mother(Nirmala, SPEAKER)

Symbolic Output: “mother” (Confidence: 1.0), Correct.

With coreference resolution active (Algorithm 1), the pronoun *her* is resolved to Desya, yielding a fully grounded fact base. The Horn-clause engine then derives the answer deterministically in three steps.

Hybrid Answer: “mother” (Confidence: 1.0), Correct

Analysis: This case demonstrates the critical role of coreference resolution (Algorithm 1). Without it, the parser produces the unresolved fact Daughter(Nirmala, *her*), which cannot participate in forward chaining, causing the symbolic engine to fail and triggering a neural fallback that predicts “daughter” with low confidence (0.31). With coreference resolution active, the full reasoning chain is recovered and the correct answer is derived deterministically. The quantitative impact of this component is confirmed in Table 6: disabling coreference resolution reduces overall accuracy by 7.4 percent.

4.3.4 Case Study 4: Seven-Hop Reasoning Chain

Input Story:

“Apollo is the father of Hermes. Zeus is the father of Apollo. Cronus is the father of Zeus. Uranus is the father of Cronus. Gaia is the mother of Uranus. Rhea is the mother of Zeus. Metis is the mother of Apollo.”

Query: “What is Gaia to Hermes?”

Neural Prediction: “great-grandmother” (Confidence: 0.58), Incorrect

Symbolic Inference Steps (7-Hop Chain, Algorithm 2):

- | | |
|--|-----------------------------|
| 1. Father(Apollo, Hermes) | [Direct] |
| 2. Father(Zeus, Apollo) | [Direct] |
| 3. Father(Cronus, Zeus) | [Direct] |
| 4. Father(Uranus, Cronus) | [Direct] |
| 5. Mother(Gaia, Uranus) | [Direct] |
| 6. Grandfather(Uranus, Zeus) | [Father + Father] |
| 7. Great_Grandfather(Uranus, Apollo) | [Grandfather + Father] |
| 8. Great_Great_Grandfather(Uranus, Hermes) | [Composition Rule] |
| 9. Great_Great_Grandmother(Gaia, Hermes) | [Mother + Composition Rule] |

Hybrid Answer: “great-great-grandmother” (Confidence: 1.0), Correct

Analysis: This seven-hop chain traces the path Gaia → Uranus → Cronus → Zeus → Apollo → Hermes through five intermediate entities. The neural model’s prediction of “great-grandmother” reflects the well-documented accuracy collapse at

Table 6 Impact of coreference resolution (Algorithm 1) on the hybrid system, evaluated on the CLUTRR test set [5]

Configuration	Accuracy	F1-Score
Without Coreference	87.3%	0.851
With Coreference	94.7%	0.923

more than 7 hops (Table 4: 62.1%): the training distribution contains few examples of 7-hop chains, so the model defaults to the shallower pattern that dominates the training set. The symbolic module’s saturation-based forward chaining applies each composition rule in sequence, mechanically expanding the knowledge graph until the target relation is found. The depth-invariant character of this process and identical algorithmic behaviour regardless of chain length—is precisely what distinguishes the symbolic approach from statistical pattern matching.

4.4 Ablation Studies

4.4.1 Impact of Coreference Resolution

Table 6 presents a controlled comparison of the hybrid system with and without the coreference resolution preprocessing step (Algorithm 1). When coreference resolution is disabled, all pronouns in extracted facts are left as raw pronoun tokens (e.g., *her*, *my*, *him*) rather than being substituted with their named referents. This causes the symbolic parser to fail on any sentence that uses pronouns to refer to previously introduced entities, a common pattern in CLUTRR narratives. When the parser fails to extract a complete fact chain, the orchestrator falls back to the neural module, which incurs the 73.2% logical consistency penalty. Enabling coreference resolution restores these linkages, allowing the symbolic engine to construct complete knowledge graphs and perform deterministic forward chaining. The impact is substantial: accuracy improves from 87.3% to 94.7% (+7.4 pp) and F1-score from 0.851 to 0.923 (+0.072), confirming that pronoun resolution is a critical architectural requirement for any symbolic system operating over natural language input.

4.4.2 Impact of Specificity Ranking

Table 7 compares two answer selection strategies for cases where the forward chaining engine derives multiple valid facts that all match the target query. The *first-match* strategy returns the first matching fact encountered during forward chaining, with no preference between specific and generic relation labels. This fails when both Brother(Luke, Ben) and Sibling(Luke, Ben) are derived, because the ground-truth annotation always uses the most specific label, and first-match may return the generic “sibling” label depending on rule firing order. The *specificity ranking* strategy avoids this by scoring all valid derived facts by the specificity hierarchy defined in Section 3.2 and returning the highest-scoring one. Specificity ranking improves accuracy from 91.2% to 94.7% (+3.5 pp) and F1 from 0.892 to 0.923 (+0.031).

Table 7 Impact of specificity ranking on the hybrid system, evaluated on the CLUTRR test set [5]

Configuration	Accuracy	F1-Score
First Match	91.2%	0.892
Specificity Ranking	94.7%	0.923

5 Discussion

5.1 Behaviour of the Symbolic Model

The symbolic module consistently exhibits depth-invariant accuracy across all inference chain lengths (Table 4, Fig. 5). Performance does not decline as the number of hops increases since each inference step is controlled by a deterministic Horn-clause rule [1] rather than a learned statistical correlation. In contrast to the neural model’s 33.1 percent collapse, the symbolic module achieves 98.1% accuracy on 2-hop chains and retains 93.2% accuracy at more than 7 hops—a loss of just 4.9 percent. The module’s primary failure mechanism is parser failure: when the natural language input contains syntactic structures not covered by the 25 regex patterns, no facts are extracted, the knowledge graph stays empty, and the module returns UNKNOWN. This brittleness at the perception stage is the defining drawback of the purely symbolic approach and the main driving force for the hybrid design.

5.2 Strengths of the Hybrid Approach

The experimental results in Tables 3–7 and Figs. 4–5 show that combining neural and symbolic components yields capabilities that neither paradigm achieves alone. Assigning all multi-hop inference to the deterministic symbolic module preserves compositional generalization, while the neural fallback provides linguistic resilience when the parser fails. The hard symbolic-priority rule in Algorithm 3 provides a structural guarantee of 100% logical consistency on symbolically tractable queries; this is not a statistical average that may deteriorate under distributional shift [9]. For each symbolically resolved inquiry, the system produces human-readable proof traces, offering the step-by-step auditability that the interpretability literature recognizes as crucial for AI deployment in high-stakes domains [8]. The 47 Horn clause rules encapsulate compositional knowledge that generalizes to arbitrary chain depths without requiring a single training example of those depths [1, 10], offering an additional level of sample efficiency unmatched by purely statistical techniques.

5.3 Limitations and Challenges

The most important drawback is the brittleness of the regex-based fact extractor: sentences that use constructions not covered by the 25 defined patterns force the system to rely on the neural module, resulting in a quantifiable accuracy cost of 7.4 percent (Table 6) and the loss of the logical consistency guarantee for those queries.

The present 47 rules fully encode the CLUTRR kinship ontology, however they cannot be easily expanded to additional domains without skilled knowledge engineering. The scalability of forward chaining (Algorithm 2) is also a real consideration for deployment at scale, since the number of rule firings per iteration increases quadratically with the size of the fact set. Finally, the heuristic coreference resolution (Algorithm 1) fails when dealing with complicated discourse structures involving several concurrent referential chains or long-distance anaphora.

5.4 Comparison with Related Work

On the CLUTRR benchmark [5], the hybrid system performs noticeably better than end-to-end neural approaches: neural models drop from 95% at 2-hop chains to 62% at more than 7 hops, while the suggested system maintains 92.4% at the same depth, consistent with the compositional generalization failures reported by Marcus [7] and Lake and Baroni [6]. Compared to NTPs [11], the suggested method sacrifices end-to-end differentiability in favor of interpretability and computational efficiency. Differentiable methods like DeepProbLog [12] or LTNs [14] relax logical constraints into continuous functions that allow mild violations, and therefore cannot achieve the 100% logical consistency of the suggested system (Table 3). In domains such as medical genealogy, legal kinship adjudication, and automated verification, where logical precision and explainability are crucial, this trade-off is acceptable [8, 9].

6 Conclusion and Future Work

This study examined multi-hop reasoning in neural, symbolic, and hybrid models using the CLUTRR benchmark [5]. By treating a refined BERT encoder [2] as a natural language perception module and allocating systematic multi-hop inference to a deterministic Horn-clause engine [1] operating over a structured knowledge graph, the proposed Hybrid Reasoner shows that high accuracy and guaranteed logical consistency are simultaneously achievable. The three main conclusions are: neural models show significant accuracy degradation with increasing inference depth (from 95.2% at 2 hops to 62.1% at more than 7 hops); symbolic reasoning maintains nearly constant accuracy regardless of depth (93.2%–98.1%) however, it requires structured input; and the hybrid approach achieves the best overall performance (94.7% accuracy and 100% logical consistency on symbolically tractable queries).

The most important short-term improvement would be the integration of ILP [1, 13] to automatically learn Horn clause rules from annotated relational data—replacing the manually generated rule library with rules induced directly from examples. This would remove the fundamental scalability barrier of the current design and eliminate the need for expert knowledge engineering when expanding to other relational domains. Replacing the regex-based fact extractor with a neural semantic parser trained on structured supervision would address the 7.4 percent accuracy penalty from parser failures, handling a far larger range of linguistic constructions while maintaining the deterministic nature of downstream symbolic inference.

Two additional improvements would expand the system’s reasoning capacity. Adding temporal linkages and event sequencing to the knowledge graph would enable

applications such as clinical timeline analysis and legal event ordering, where the arrangement of facts is just as crucial as their substance. Including probabilistic logic programming [12] as a systematic framework for treating uncertainty would enable robust reasoning over contradictory or inadequate evidence, supplementing the current emphasis on deterministic accuracy with a calibrated treatment of ambiguity. Extending Hybrid Reasoner to multi-modal inputs—integrating textual narratives with visual scene graphs, structured databases, or medical imaging data—via a modular perception front-end that extracts structured facts from non-textual modalities and feeds them into the current symbolic inference engine would greatly expand the system’s applicability to richer information sources.

Declarations

No funding was received for conducting this study.

The authors have no competing interests to declare that are relevant to the content of this article.

All authors contributed equally to the conception and design of this work. The methodology, software development, and formal analysis were carried out collectively by all authors. The original draft was prepared jointly, and all authors participated in reviewing and editing subsequent versions. All authors read and approved the final manuscript.

The CLUTRR benchmark dataset used in this study is publicly available at <https://github.com/facebookresearch/clutrr> [5]. No additional datasets were generated or analysed during this study.

References

- [1] Muggleton S, De Raedt L (1994) Inductive logic programming: Theory and methods. *Journal of Logic Programming* 19-20:629-679
- [2] Devlin J, Chang MW, Lee K, Toutanova K (2019) BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pp 4171-4186
- [3] Scarselli F, Gori M, Tsoi AC, Hagenbuchner M, Monfardini G (2009) The graph neural network model. *IEEE Transactions on Neural Networks* 20(1):61-80
- [4] Santoro A et al (2017) A simple neural network module for relational reasoning. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol 30, pp 4967-4976
- [5] Sinha K, Sodhani S, Dong J, Pineau J, Hamilton WL (2019) CLUTRR: A diagnostic benchmark for inductive reasoning from text. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*

- and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp 4506-4515
- [6] Lake BM, Baroni M (2018) Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In: Proceedings of the 35th International Conference on Machine Learning (ICML), pp 2879-2888
 - [7] Marcus G (2018) Deep learning: A critical appraisal. arXiv preprint arXiv:1801.00631
 - [8] Lipton ZC (2018) The mythos of model interpretability. Communications of the ACM 61(10):36-43
 - [9] Ribeiro MT, Singh S, Guestrin C (2016) “Why should I trust you?”: Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 1135-1144
 - [10] Towell GG, Shavlik JW (1994) Knowledge-based artificial neural networks. Artificial Intelligence 70(1-2):119-165
 - [11] Rocktäschel T, Riedel S (2017) End-to-end differentiable proving. In: Advances in Neural Information Processing Systems (NeurIPS), vol 30, pp 3788-3800
 - [12] Manhaeve R, Dumančić S, Kimmig A, Demeester T, De Raedt L (2018) DeepProbLog: Neural probabilistic logic programming. In: Advances in Neural Information Processing Systems (NeurIPS), vol 31, pp 3749-3759
 - [13] Evans R, Grefenstette E (2018) Learning explanatory rules from noisy data. Journal of Artificial Intelligence Research 61:1-64
 - [14] Serafini L, d’Avila Garcez A (2016) Logic tensor networks: Deep learning and logical reasoning from data and knowledge. In: Proceedings of the Workshop on Neural-Symbolic Learning and Reasoning (NeSy)
 - [15] Loshchilov I, Hutter F (2019) Decoupled weight decay regularization. In: Proceedings of the 7th International Conference on Learning Representations (ICLR)
 - [16] Zeng L, Liu Y, Han G, Zu-Guo Y, Liu Y (2025) ProGraphTrans: Multi-modal dynamic collaborative framework for protein representation learning. BMC Biology 23:1-15
 - [17] Shui Y, Ge X, Cao C, Wang J, Hu J, Liu Y (2025) BiMA-DTI: A bidirectional Mamba-Attention hybrid framework for enhanced drug-target interaction prediction. BMC Biology 23:1-20
 - [18] Ren J, Tang T, Jia H, Xu Z, Fayek H, Li X, Ma S, Xu X, Xia F (2025) Foundation models for anomaly detection: Vision and challenges. AI Magazine 46(4)